

Block 2

Elasticity and scalability

- 2.1 Replication problems
- 2.2 Sharding and state machines
- 2.3 DHT and publish/subscribe
- 2.4 Scalability in the cloud

2.1 Replication problems

A system stores several copies of the same data. Copies answer faster and survive failures. Every write must then reach every copy, and the rule chosen to do that fixes what a client observes.

CONTENTS

- 01 Why copies exist
- 02 The lost update
- 03 Answer 1: primary copy
- 04 Synchronous propagation
- 05 Asynchronous propagation
- 06 Answer 2: locks
- 07 Answer 3: optimistic control
- 08 Answer 4: quorum
- 09 Conflicting writes
- 10 What the client observes

A copy near the reader answers in 5 ms instead of 250 ms

Speed

Tokyo to a server in Madrid: 250 ms round trip. Tokyo to a copy in Tokyo: 5 ms.

Survival

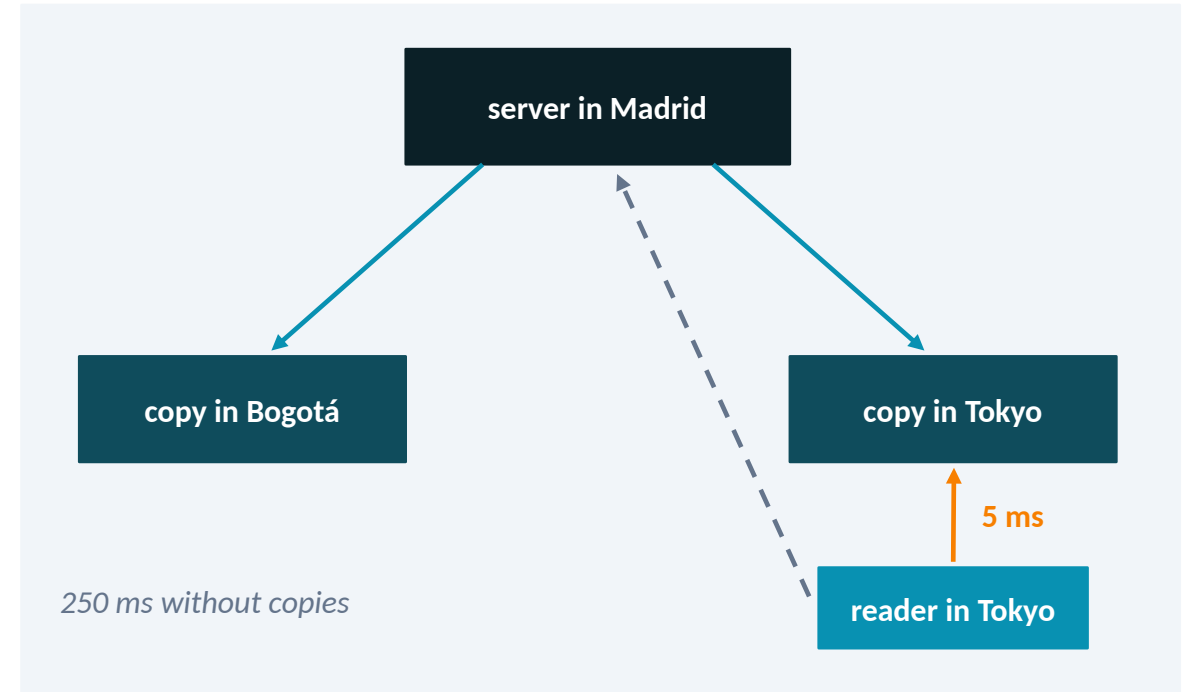
If Madrid stops, Bogotá and Tokyo keep answering. With one copy, one failure stops the service.

Read load

One read touches one copy, so ten copies serve ten times more reads per second.

The duty that copies create

Every write must reach every copy. Two copies holding different values answer the same question differently.



Every write is copied to Bogotá and to Tokyo. A video service stores a copy of each film inside the network of every operator.

Copies reduce the read delay and add one duty: every write must reach every copy.

Two clients that read before either writes lose one update

Given: B holds 200 euros. Transactions T and U each add 10 per cent to B, and neither waits for the other. Asked: the value stored at the end.

Step	Transaction T	Transaction U	Stored in B
1	reads B: 200		200
2		reads B: 200	200
3	computes $200 \times 1.10 = 220$, writes 220		220
4		computes $200 \times 1.10 = 220$, writes 220	220

The result

Stored 220, correct $200 \times 1.10 \times 1.10 = 242$. B was raised twice and kept one rise.

Why

Both read 200 before either wrote, so the second write replaced the first.

The same failure between copies

Two devices editing one basket; two payments on one account in the same second.

What is needed

An order for the writes, or a check that the value did not change between the read and the write. Four answers follow.

A lost update produces no error: the stored value is legal and wrong.

ANSWER 1: ONE MACHINE ORDERS THE WRITES

Primary copy: every write crosses one machine, backups serve reads

Goal

Give every write one order, so that no update is lost.

Structure

One primary and N backups, all holding the same data.

Steps

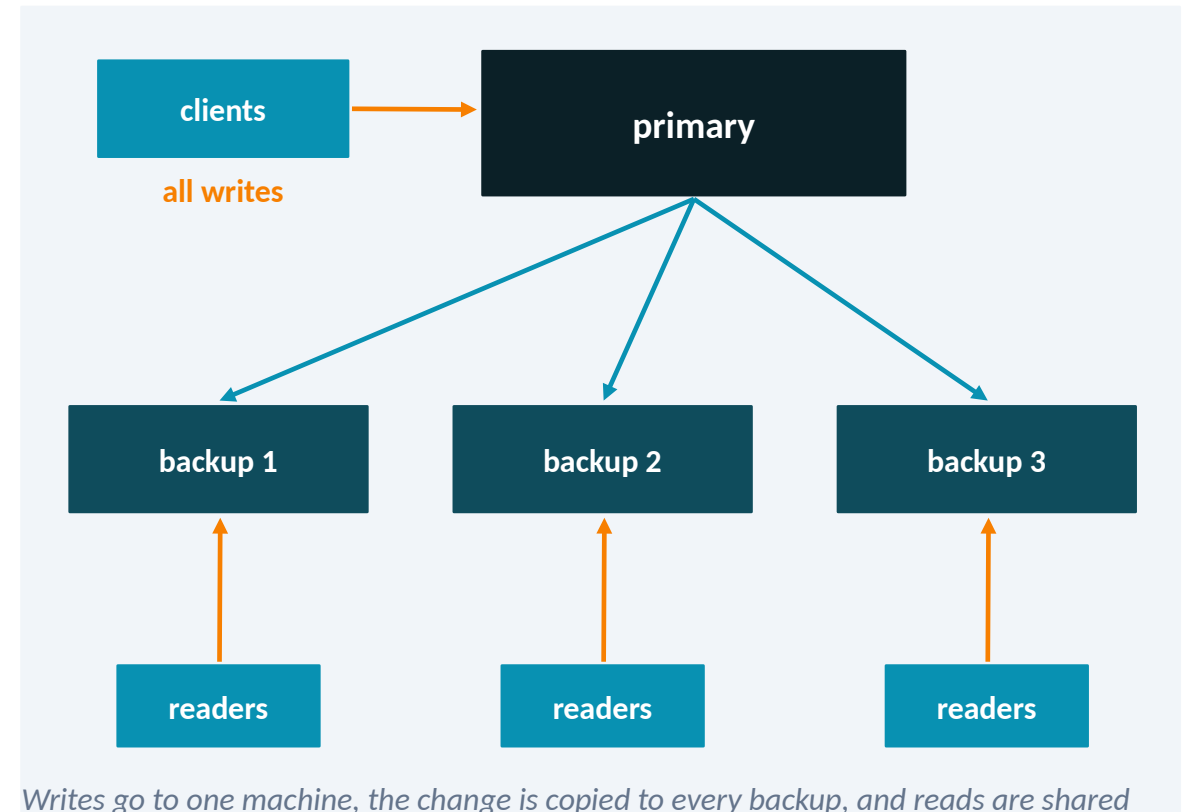
- 1 the client sends the write to the primary.
- 2 the primary applies it and fixes its position in the order.
- 3 the primary sends the change to every backup.
- 4 reads go to any backup.

Limit

Every write crosses one machine, so the write rate is that of one machine. If the primary stops, one backup is promoted.

Real system

PostgreSQL and MySQL: one primary and several read replicas.

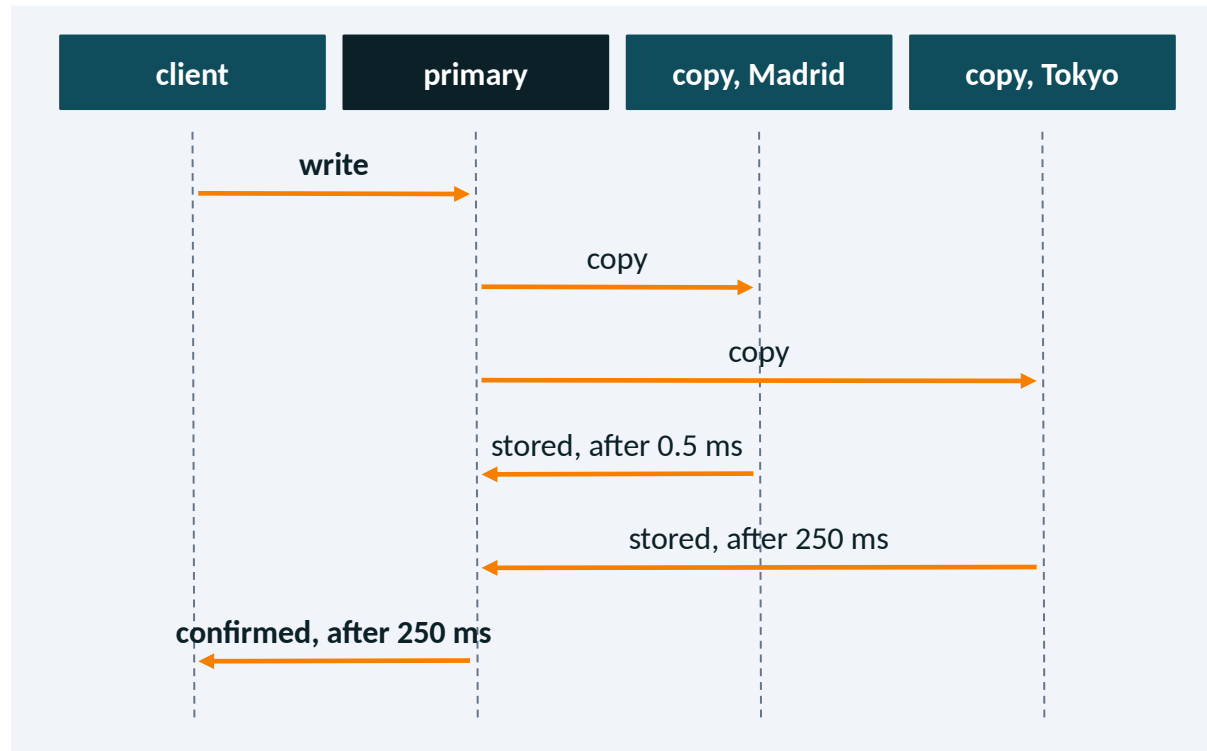


Writes go to one machine, the change is copied to every backup, and reads are shared between the backups.

Backups multiply the read capacity; every write still crosses one machine, so writes do not scale.

STEP 3 OF THE PRIMARY, FIRST OPTION

Synchronous propagation confirms after the slowest copy



The confirmation waits for the copy in Tokyo.

Rule

The primary confirms to the client only when every copy has stored the write.

Cost, with numbers

Madrid stores it after 0.5 ms, Tokyo after 250 ms. The client waits for the slowest: 250 ms per write.

What the client gains

Any copy read after the confirmation holds the new value: no moment at which two copies disagree.

What fails

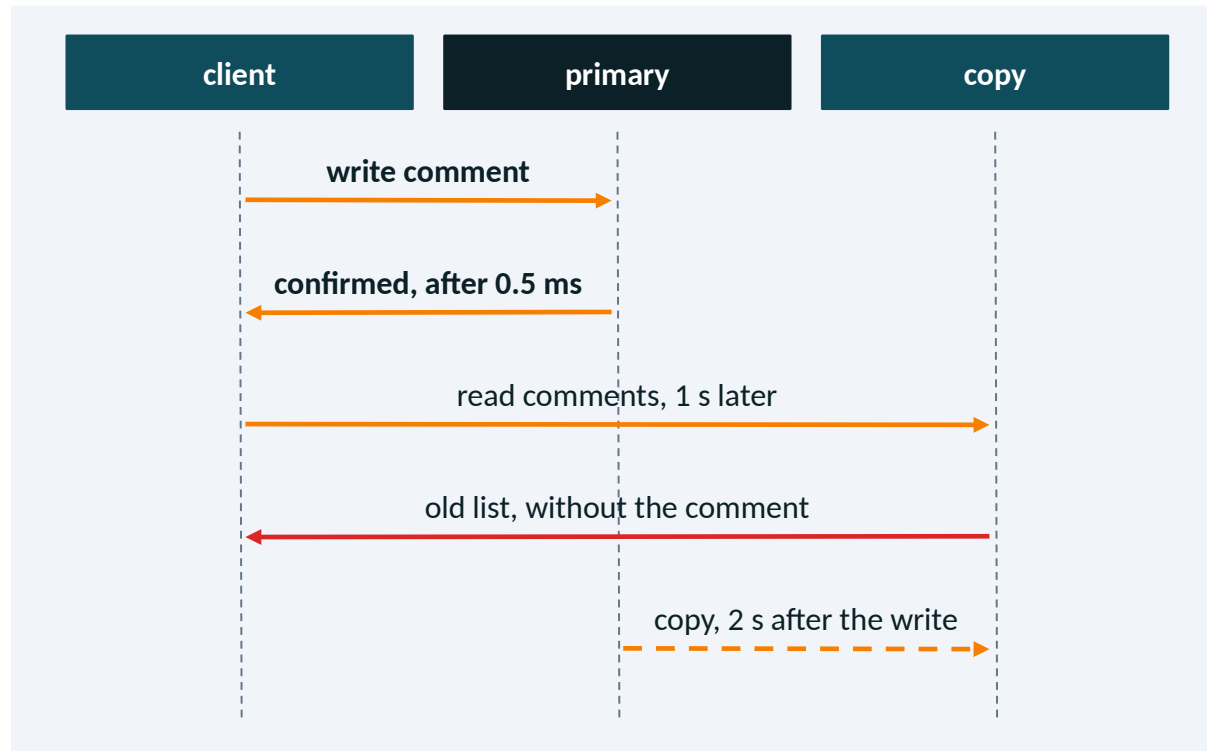
One unreachable copy blocks every write. Each copy added is one more machine that can block.

Use

A bank transfer, because a movement lost in one copy is not acceptable.

Every read after the confirmation returns the new value, and every write costs the round trip to the slowest copy.

Asynchronous propagation confirms in 0.5 ms and leaves a lag



The copy answered a read before it received the change.

Rule

The primary confirms at once and sends the change afterwards.
Replication lag: the time until a copy holds it, milliseconds in a quiet system and seconds under load.

Visible symptom

A user posts a comment, refreshes, and the comment is gone: the read was served by a copy that was behind.

Three repairs

- 1 read from the primary during a few seconds after a write.
- 2 keep each client on one copy.
- 3 the client sends the position it has already seen, and a copy behind it does not answer.

Risk

If the primary fails before sending the change, the write is lost.

A write costs one local round trip, and a read may return a value as old as the lag.

ANSWER 2: MAKE THE SECOND WRITER WAIT

Locks: the second writer waits; opposite order gives a deadlock

Lock

A mark on a data item. The holder uses it and the others wait. A read lock is shared; a write lock is exclusive.

Steps on B

- 1 T takes the write lock on B.
- 2 U asks for it and waits.
- 3 T writes 220 and releases the lock.
- 4 U reads 220 and writes 242.

Deadlock

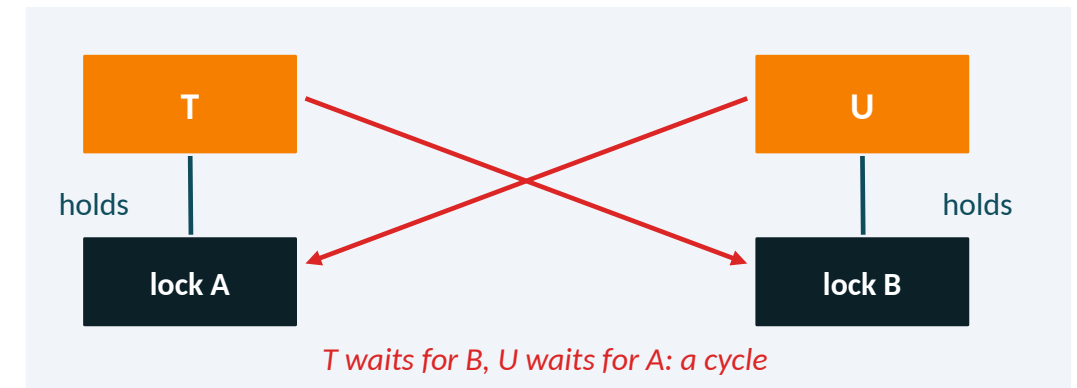
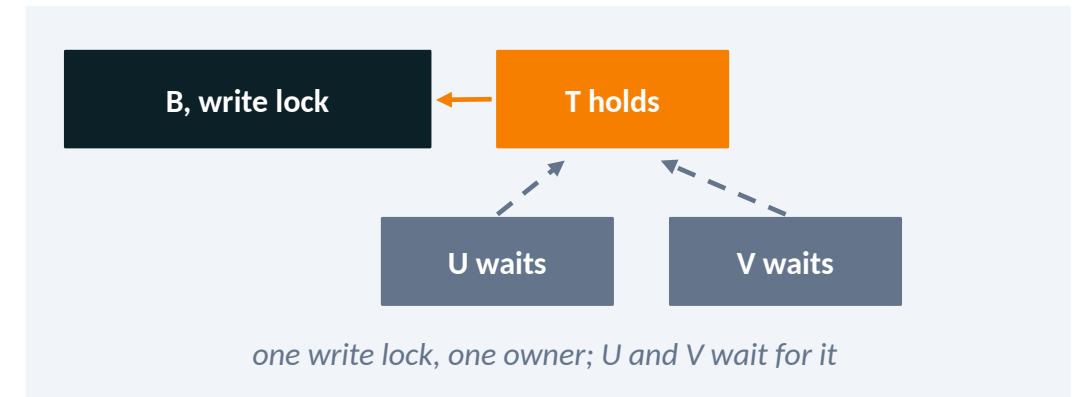
T holds A and asks for B; U holds B and asks for A. Neither continues. Detection: a cycle in the graph of who waits for whom.

Resolution

A timeout aborts one transaction, releases its locks and starts it again.

Cost

Waiting. Throughput falls as more transactions ask for the same items.



A lock removes the lost update and adds waiting. A cycle of waits is a deadlock, and one transaction is aborted to break it.

ANSWER 3: NO WAITING, CHECK BEFORE WRITING

Optimistic control writes only if the version is still the one read

Given: B holds 200 euros with version 7. T and U each add 10 per cent. Every write names the version it read, and the store accepts the write only if that version is still the current one.

Step	Transaction T	Transaction U	Stored value, version
1	reads B: 200, version 7	reads B: 200, version 7	200, v7
2	writes 220 if version is 7: accepted		220, v8
3		writes 220 if version is 7: rejected, current is 8	220, v8
4		reads again: 220, v8. Computes $220 \times 1.10 = 242$	220, v8
5		writes 242 if version is 8: accepted	242, v9

Steps

- 1 read the value and its version.
- 2 compute on a private copy, with no lock.
- 3 write only if the version is unchanged.
- 4 if rejected, return to 1. Stop when accepted.

When it wins and when it loses

Wins with many readers and few writers: nobody waits. Loses with frequent conflicts: the work is repeated.

Real system

The HTTP ETag header: the browser sends the version it read and the server rejects a stale update.

Optimistic control never waits; a transaction whose check fails repeats its work with the current value.

Quorum: $W + R > N$ makes every read meet the latest write

Three numbers

N copies. W copies must store a write before it is confirmed. R copies must answer a read.

Steps

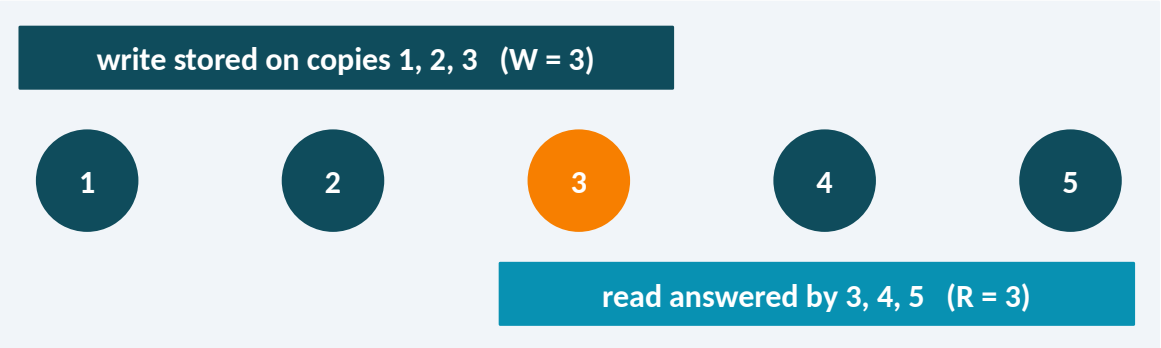
Write: send to the N copies, confirm after W answers.
Read: ask the copies, wait for R answers, keep the newest version.

Why the rule holds

$W + R$ copies do not fit in N without sharing one, so the shared copy holds the latest write.

Real system

Cassandra and DynamoDB let the client choose W and R on every operation.



$N = 5, W = 3, R = 3$: $3 + 3 = 6 > 5$, so copy 3 is in both sets and holds the write.

W	R	$W + R > 5?$	What the client observes
3	3	$6 > 5$, yes	the latest value; each operation contacts 3 copies
1	1	$2 > 5$, no	a copy that missed the write may answer: old value
5	1	$6 > 5$, yes	fast reads; one copy down blocks every write
2	3	$5 > 5$, no	the two sets can be disjoint: old value possible

With $W + R > N$ the set that stored the write and the set that answers the read always share at least one copy.

WHEN $W + R$ IS NOT ABOVE N , OR A DEVICE IS OFFLINE

Conflicting writes: last writer wins, keep both, or merge

How it happens

With no primary, two clients write different values to two copies at the same moment, or while one client is offline. Both writes are accepted.

Last writer wins

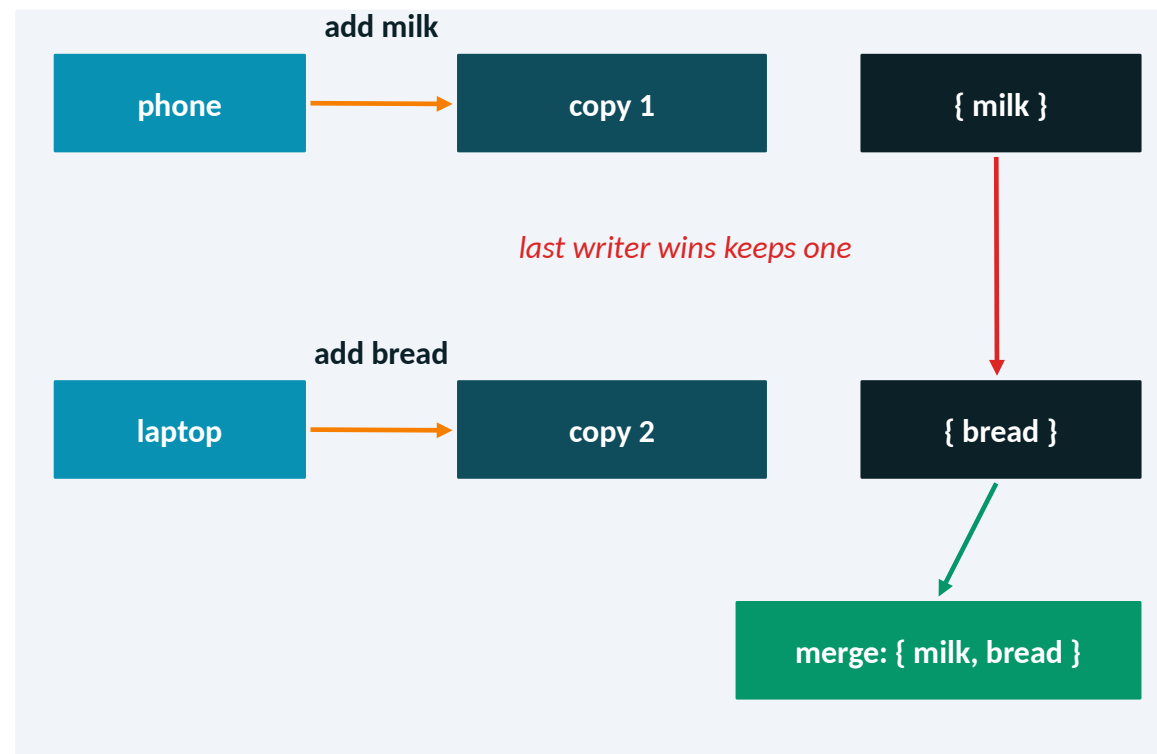
Each write carries a timestamp and the later one is kept. Simple, and it silently discards the other change.

Keep both

The store returns the two values on the next read and the application chooses. Nothing is lost; the application needs code.

Merge by data type

For a set the merge is the union, so adding items on two copies never loses one. Counters and lists have merges defined this way.



A shopping basket edited on two devices while one of them is offline.

Two copies can receive different writes; the rule that reconciles them fixes what the user loses.

Five arrangements and the value the client observes

Arrangement	The client observes	The cost
One copy	always the latest write	no survival, no growth
Primary with synchronous copies	always the latest write	every write waits for the slowest copy
Primary with asynchronous copies	possibly an old value on a read served by a copy	writes are fast, reads can be stale
Quorum with $W + R > N$	always the latest write	every operation contacts several copies
Quorum with $W + R \leq N$	the copies agree after some time	fastest, conflicting writes must be reconciled

Relational database

One primary, asynchronous read replicas: fast writes, reads can be stale. PostgreSQL.

Wide column store

No primary, quorum chosen per operation: the application picks speed or correctness. Cassandra.

Content delivery network

Read only copies with an expiry time: very fast reads, a change takes minutes. Akamai.

The arrangement of the copies fixes the answer a client can observe; each row is a different choice of cost.

BLOCK 2. ELASTICITY AND SCALABILITY

2.2 Sharding and state machines

Copies repeat the data, so they do not raise the storage. When the data no longer fits on one machine it is divided into pieces. One rule places each key, a second rule keeps the copies of each piece identical.

CONTENTS

- 01 Copies against division
- 02 Range and hash partition
- 03 Hot spots
- 04 Hash mod N breaks on growth
- 05 Consistent hashing
- 06 Routing a request
- 07 Shards with copies
- 08 Replicated state machine
- 09 The log

COPIES DO NOT RAISE THE STORAGE

Sharding: each machine holds a part, so storage and writes grow

Shard

A piece of the data held by one machine. Every key is stored in exactly one shard.

Why copies are not enough

Every copy holds the whole data, so the storage limit is the disk of one machine and every write is applied on every copy.

The number that settles it

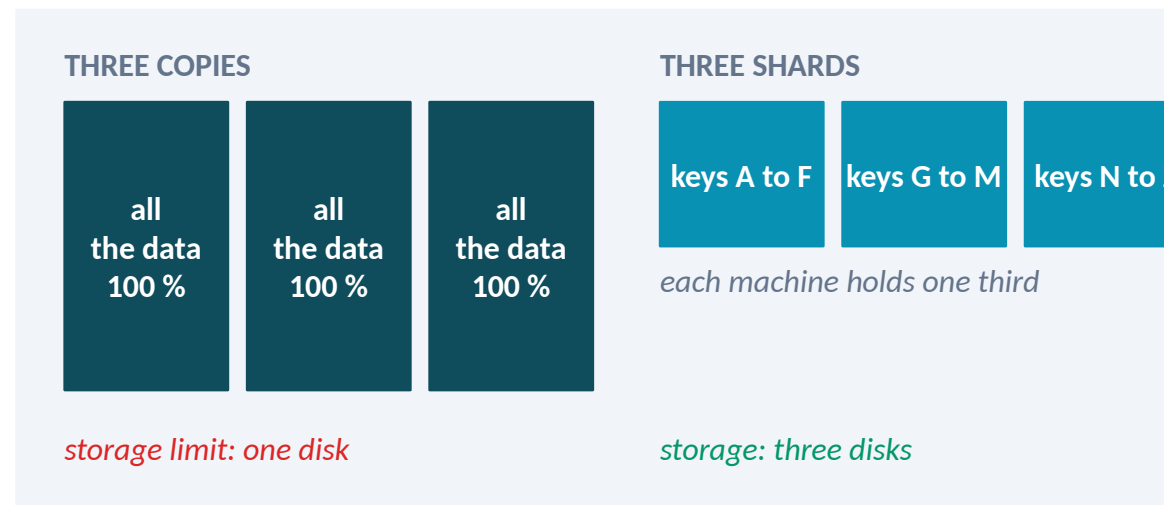
10^{11} messages per day at 100 bytes each is 10 terabytes per day: no single disk holds a month.

The new question

Which shard holds key k? It did not exist with one machine. The rule that answers it comes next.

Real system

MongoDB divides a collection into shards by a key chosen by the designer.



Copies repeat the data; shards divide it.

A shard is a piece of the data held by one machine. Every request must first find its shard.

Range partition keeps order; hash partition spreads the writes

Given: nine user names on three machines. By range: A to F on node 1, G to M on node 2, N to Z on node 3. By hash: a fixed function gives each name the number shown, and the machine is $1 + (\text{number} \bmod 3)$.

Name	Ana	Beto	Carla	Diego	Hugo	Luis	Marta	Nuria	Zoe
By range	node 1	node 1	node 1	node 1	node 2	node 2	node 2	node 3	node 3
Hash number	17	22	9	31	4	26	13	8	15
Number mod 3	2	1	0	1	1	2	1	2	0
By hash	node 3	node 2	node 1	node 2	node 2	node 3	node 2	node 3	node 1

By range

Each machine takes an interval, so a query for every name between Carla and Diego touches one machine. The danger: names that arrive together fall in one interval, and log records divided by date put all of today on the last machine.

By hash

$\text{shard} = 1 + (\text{hash}(k) \bmod 3)$. Here 2, 4 and 3 names per machine, and an even spread with thousands. The price: Carla and Diego land on different machines, so a query over an interval asks every machine.

Range when the queries follow the order of the key; hash when the writes must be spread.

A hot key saturates one shard while the average looks fine

Hot spot

One shard receives far more work than the others, so the system has the capacity of that one machine.

The numbers in the figure

Five shards, 100 writes per second in total. Average 20 per shard; shard 3 receives 84 and the others 4. The average says 20 per cent of the capacity; shard 3 is saturated.

How to detect it

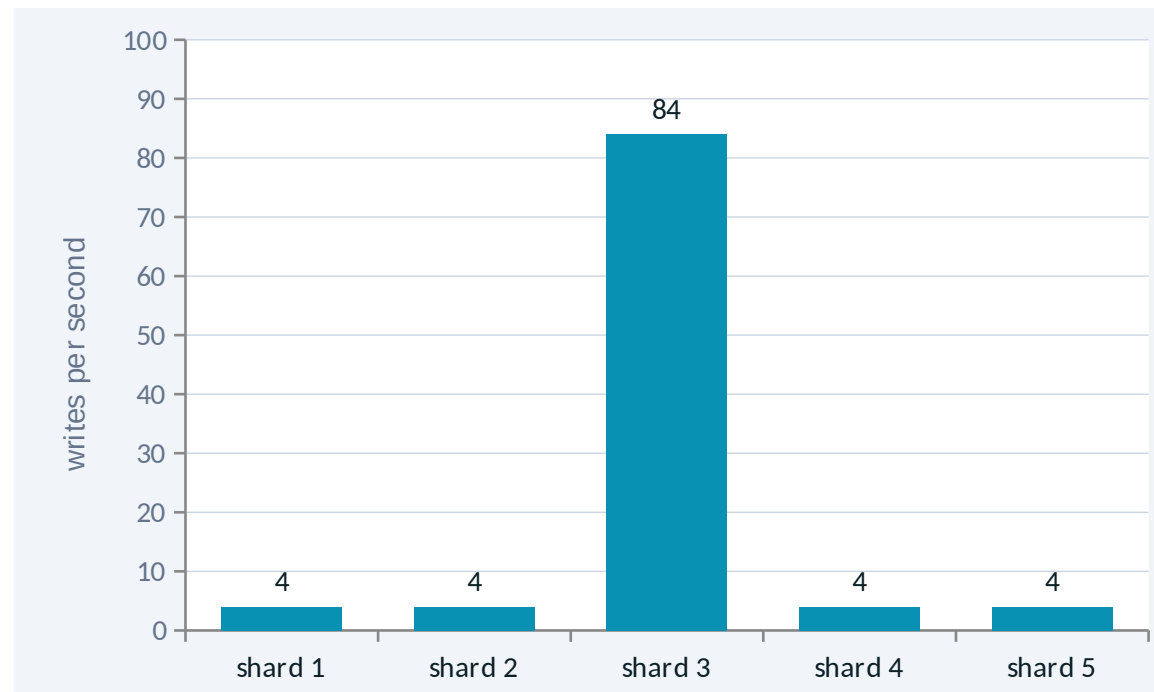
Compare the maximum shard load with the average. Here 84 against 20.

The IoT version

Two thousand sensors report at the start of every minute. With the instant of the reading as key, every reading lands on one shard.

Three repairs

- 1 key = sensor identifier.
- 2 key = instant plus a random number 0 to 9.
- 3 spread the reporting instants over the minute.



One key receives most of the traffic; an average of 20 per shard hides a shard at 84.

Measure the load of the busiest shard, never the average; choose a key that splits the traffic arriving together.

WHAT GOES WRONG WHEN A MACHINE IS ADDED

Hash mod N: growing from 3 to 4 machines moves 5 of 6 keys

Given: six keys placed by number mod N, with N machines numbered from 0. N grows from 3 to 4. Asked: how many keys change machine.

Key	Hash number	Machine with 3, number mod 3	Machine with 4, number mod 4	Moved?
Ana	17	2	1	yes
Beto	22	1	2	yes
Carla	9	0	1	yes
Diego	31	1	3	yes
Hugo	4	1	0	yes
Luis	26	2	2	no

The result

Five of six changed machine: a key stays only when its number gives the same remainder with 3 and with 4.

Why it matters

Every moved key is copied before the system answers normally: hours with terabytes.

What is needed

A rule where adding one machine among N moves about one part in N of the keys.

With hash mod N, adding one machine moves about three quarters of the keys; a rule that moves one part in N is needed.

Consistent hashing: a new machine takes only its neighbour keys

Structure

A circle of positions 0 to 99. Machines and keys are placed on it by the same hash function.

Placement rule

A key is stored by the first machine found going clockwise from the position of the key on the circle.

The example

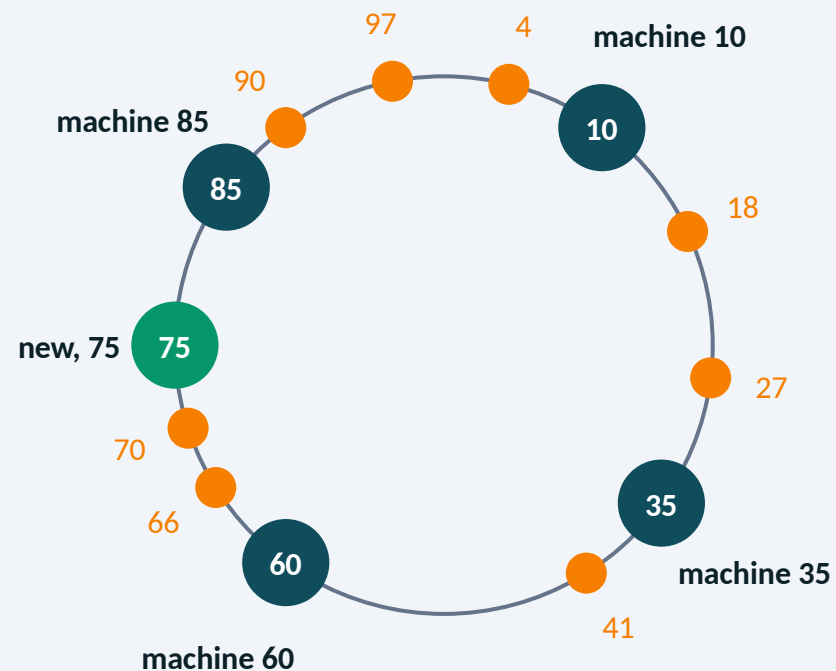
Machines at 10, 35, 60, 85. Key 4 goes to 10; 18 and 27 to 35; 41 to 60; 66 and 70 to 85; 90 and 97 go round to 10.

Adding a machine at 75

It takes the keys between 60 and 75: 66 and 70. The other six stay: 2 of 8 moved, about one part in N.

Virtual nodes

Each machine is placed at several positions, so the pieces are similar. Cassandra builds its ring this way.



A key is stored by the first machine found clockwise, so a new machine takes only the keys between its position and the previous machine.

Routing: a directory, a proxy, or the map inside the client

A directory service

A separate service stores which shard holds which keys. Every request asks it first: one extra round trip of 0.5 ms and a component that must not fail.

A proxy

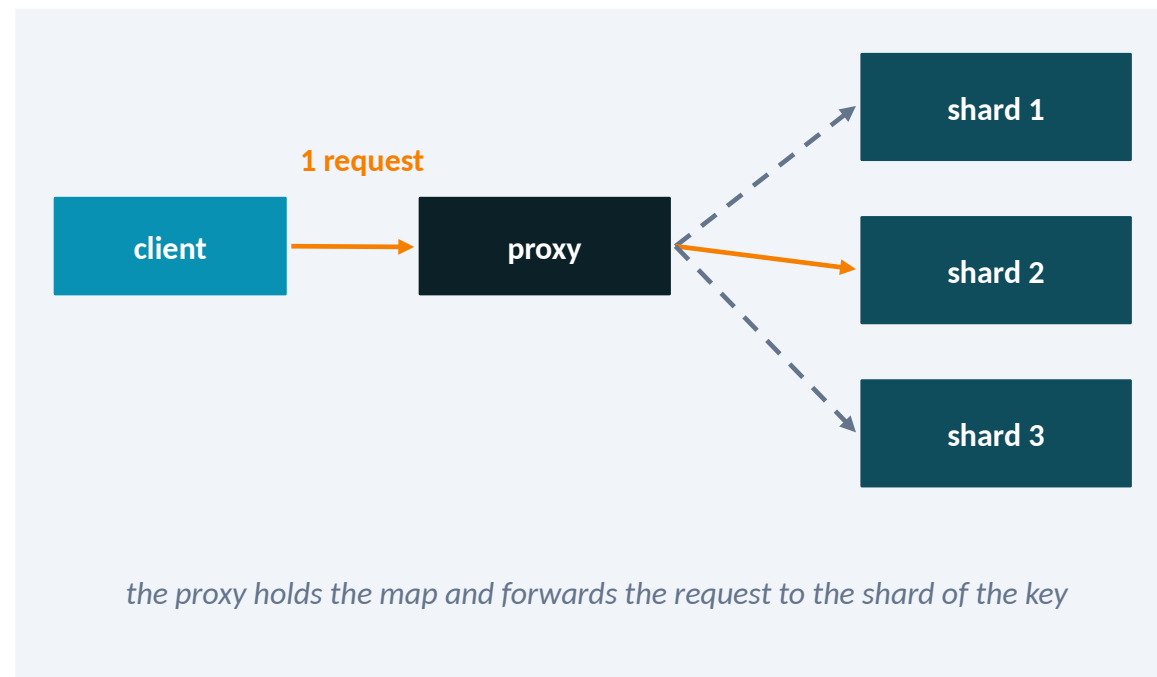
Every request goes to a proxy that forwards it to the right shard. The client stays simple; the proxy adds one hop of 0.5 ms and must scale itself.

A map in the client

The client library holds the map and contacts the machine directly: no extra hop, and every client must receive the new map when a machine joins or leaves.

Real systems

MongoDB routes through a proxy called mongos. DynamoDB gives the client library the map.



One request, one extra hop, a simple client.

Someone must hold the map from key to shard, and the map changes whenever a machine joins or leaves.

Three shards with three copies each: nine machines

Structure

Three shards, each copied three times: nine machines, each holding one copy of one shard.

What each idea solves

Dividing raises the amount of data and the write rate. Copying gives survival and more reads.

A leader per shard

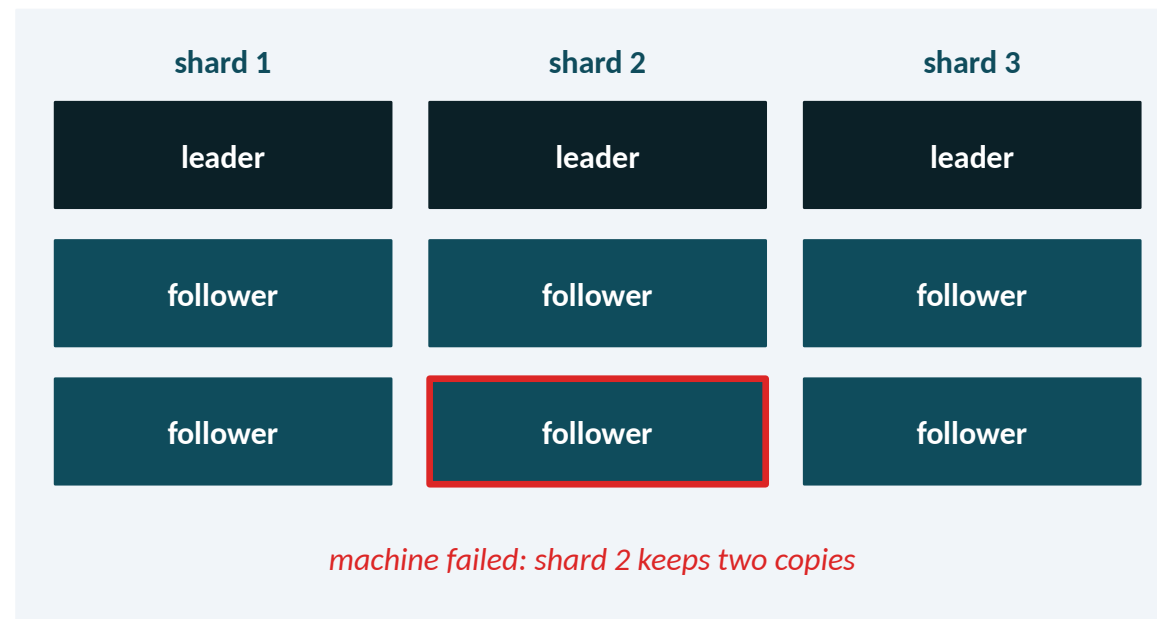
Each shard has its own primary, called leader, and two followers. The three leaders run on different machines, so writes to different shards go in parallel.

Losing one machine

It held one copy of one shard. That shard keeps two copies and every key is served. The missing copy is rebuilt on a healthy machine.

Losing every copy of a shard

Requests for its keys fail and every other key is served. One machine holding everything fails completely.



Each column is one shard with its three copies; each box is one machine.

Real system

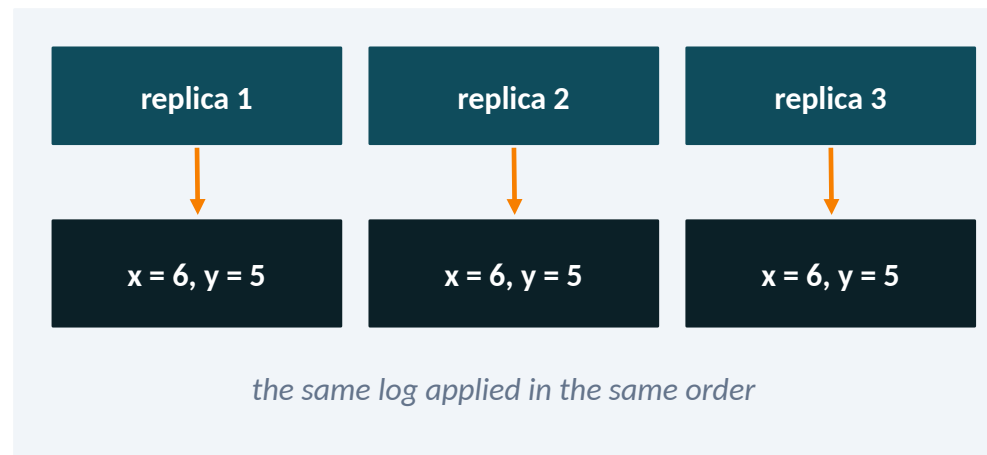
Kafka divides a topic into partitions and copies each partition three times.

Division raises data and write rate; copies give survival. Losing a machine removes one copy and the service continues.

Replicated state machine: same start, same order, same state

Definition: a replicated state machine is a set of copies that begin in the same state and apply the same operations in the same order. If every operation is deterministic, identical input in identical order gives identical state, so no copy compares its data with another.

Position in the log	Operation	State after it, on every copy
1	set x = 1	x = 1
2	add 5 to x	x = 6
3	set y = 2	x = 6, y = 2
4	add 3 to y	x = 6, y = 5



Steps

- 1 the leader of the shard appends the operation to the log and gives it a position.
- 2 every copy applies the operations in order of position.
- 3 deterministic operations give identical state, with no comparison of data.

What must be agreed

Only the order: one number per operation. Real system: etcd, the configuration store of Kubernetes.

Agreement on the order of the operations replaces agreement on the data.

The log: append, number, apply; a new copy resumes from a snapshot

Operations

Append at the end; read from a position; write a snapshot of the whole state at a position; delete the entries before the last snapshot.

Position

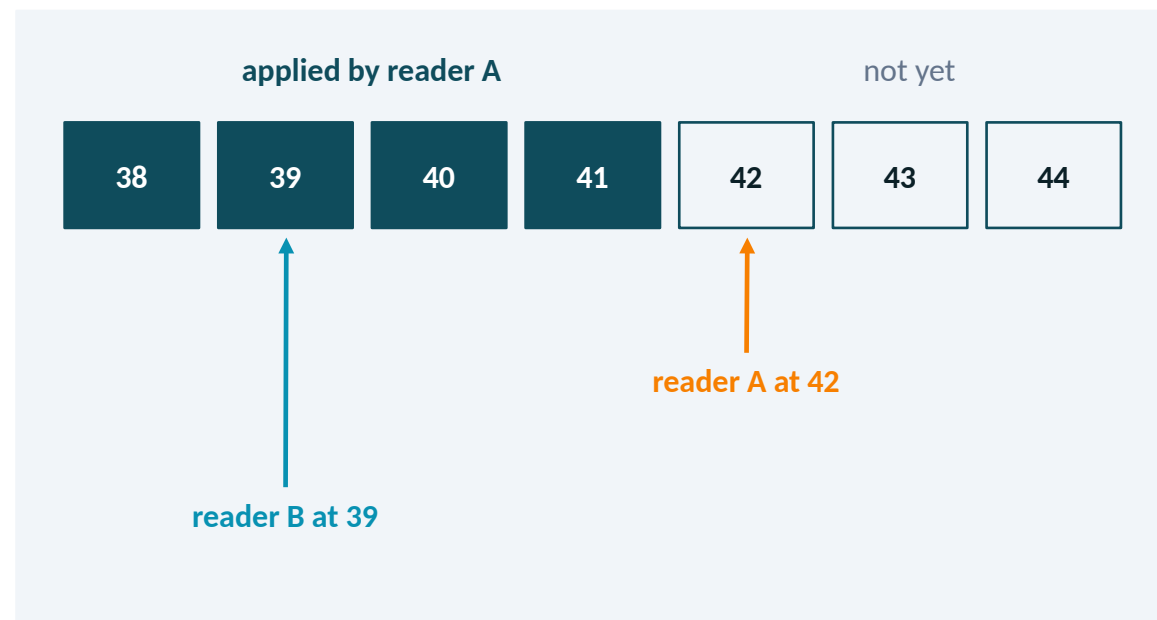
Each entry has a number. A copy that applied entry 41 asks for 42 onwards, and nothing already written changes.

Several readers

Each reader keeps its own position, so a slow reader never blocks a fast one.

Worked example

The log holds 5000 entries and the last snapshot is at 4200. A new copy loads the snapshot and applies 4201 to 5000: 800 entries instead of 5000.



Each reader keeps its own position, so they never block each other.

Real system

Kafka stores exactly this log and lets many services read it, each at its own position.

A copy that applied up to position 4200 asks for 4201 onwards: recovery is a subtraction, not a comparison of data.

BLOCK 2. ELASTICITY AND SCALABILITY

2.3 DHT and publish/subscribe

The same ring, now across machines that nobody owns: Chord finds a key in a few jumps. Then the section moves from storing data to moving messages between programs that never learn each other's address.

CONTENTS

- 01 The ring without a central map
- 02 Chord: the lookup algorithm
- 03 Chord: join, leave, copies
- 04 Message queues
- 05 The broker
- 06 Publish and subscribe
- 07 MQTT
- 08 Multicast and coupling

A distributed hash table keeps the ring and drops the central map

Definition

A distributed hash table, DHT, stores pairs of key and value across many machines. No machine holds an index of the keys.

What changes from one company

There a map from key to shard can be published. Here machines join and leave every minute and nobody publishes a map.

What stays

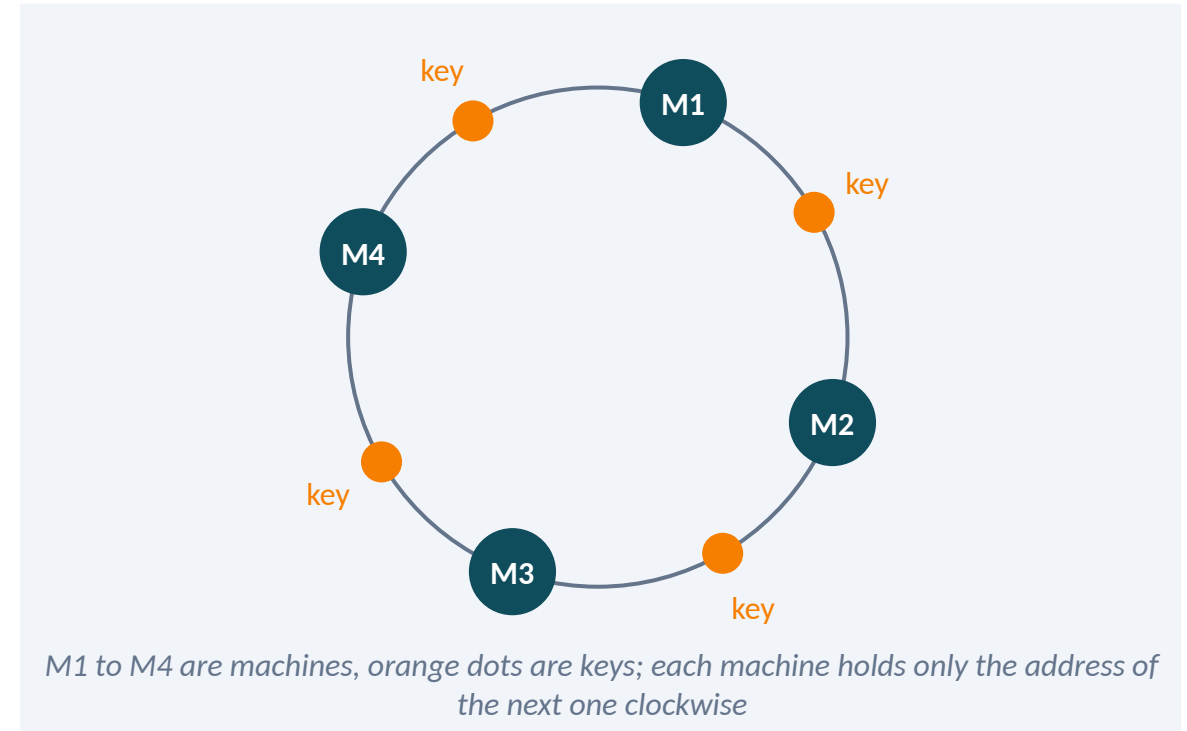
The ring of the previous section: machines and keys placed by the same hash, a key stored by the first machine clockwise.

The naive walk

Ask the next machine, which asks the next. With one million machines a lookup needs 500,000 steps on average, each one a message across the internet.

Real system

BitTorrent finds the machines that hold a file with a DHT of millions of members.



Any machine computes the position of any key; finding the owner is a walk on the ring, and the walk must be short.

Jump to the furthest shortcut that does not pass the key: $\log_2 N$ jumps

Goal

Find the machine that stores key k , starting from any machine, without a central map.

Data structure

A finger table in every machine: the addresses of the machines at distance 1, 2, 4, 8 along the ring. About $\log_2 N$ entries.

Steps

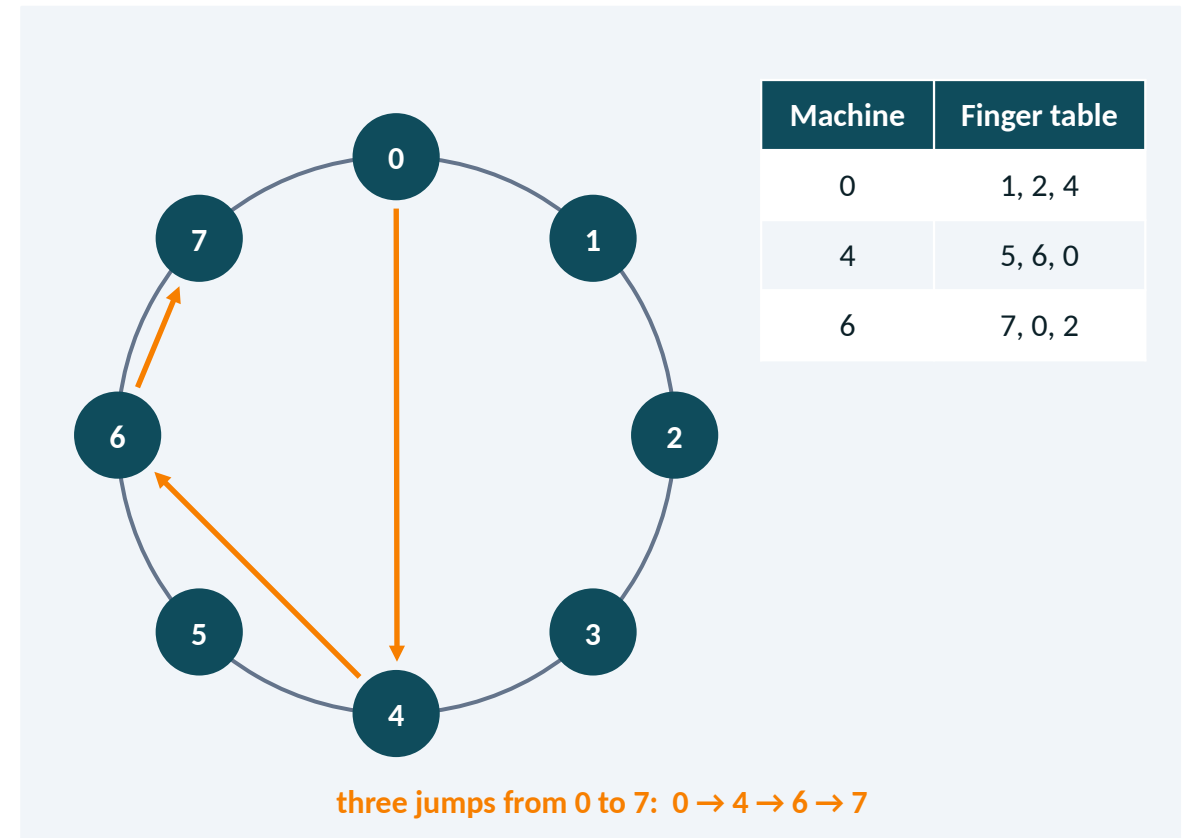
- 1 if k lies between this machine and its successor, the successor is the owner: stop.
- 2 otherwise jump to the furthest finger that does not pass k .
- 3 repeat at the machine reached.

The example, from 0 to 7

0 keeps 1, 2, 4: jump to 4. Then 4 keeps 5, 6, 0: jump to 6. Then 6 keeps 7: 7 is the owner.

Cost

$8 = 2^3$ gives 3 jumps. $1,048,576 = 2^{20}$ gives 20 jumps, with 20 addresses per machine.



Each jump at least halves the remaining distance, so N machines need about $\log_2 N$ jumps: 3 for 8 machines, 20 for a million.

Join, leave and the successor list: only neighbour keys move

A machine joins

It hashes its address, runs one lookup to find its successor, and takes the keys between the previous machine and itself.

A machine leaves politely

It hands its keys to the next machine clockwise and tells its neighbours.

A machine disappears

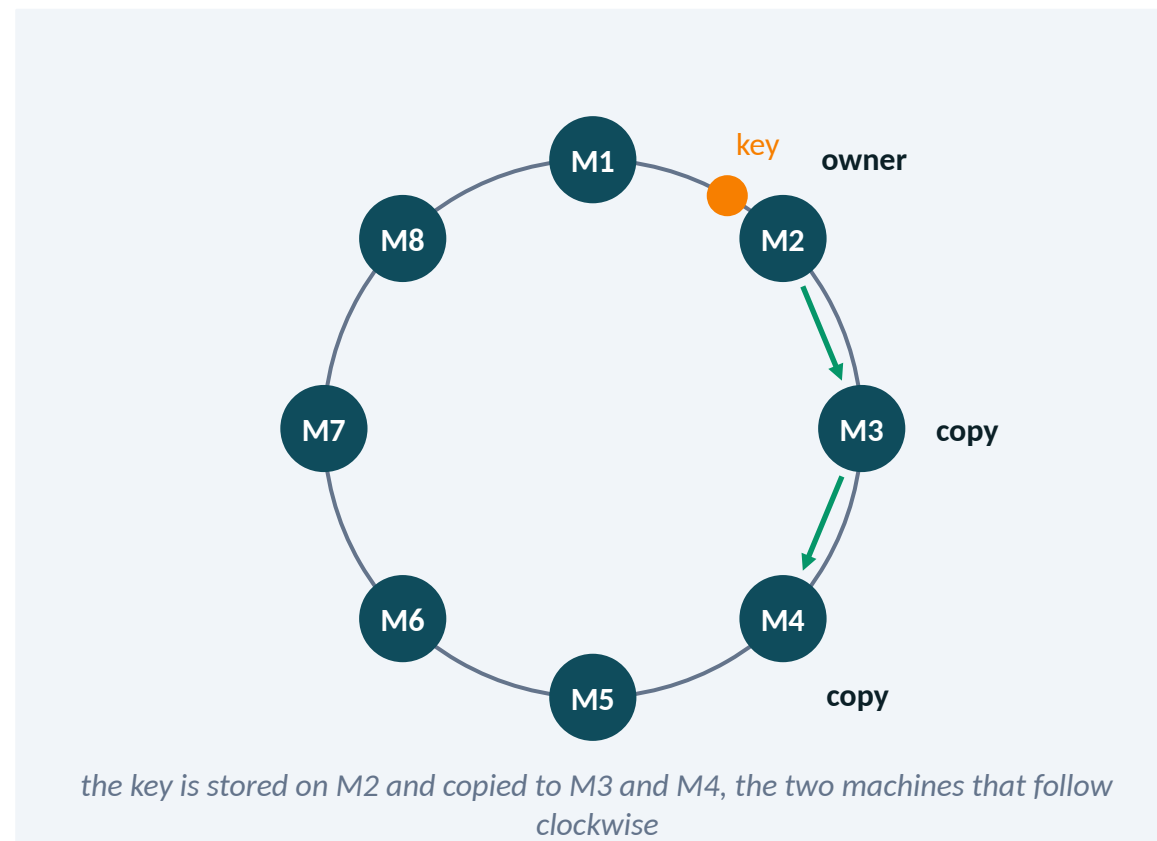
Its keys are lost unless they were copied, and in a network of home computers this happens every minute.

The successor list

Each key is stored on its owner and on the next two machines clockwise: three copies. When the owner disappears, the next machine already holds the key.

Keeping it correct

Every few seconds each machine asks its successor who its predecessor is, repairs the ring and copies the keys again.



A key is stored on its owner and on the next machines clockwise, so a machine that disappears loses no key.

A message queue stores the message until the receiver reads it

Transient messaging

The message exists only while both programs run. A socket works this way: a write with no reader at the other end delivers nothing.

Persistent messaging

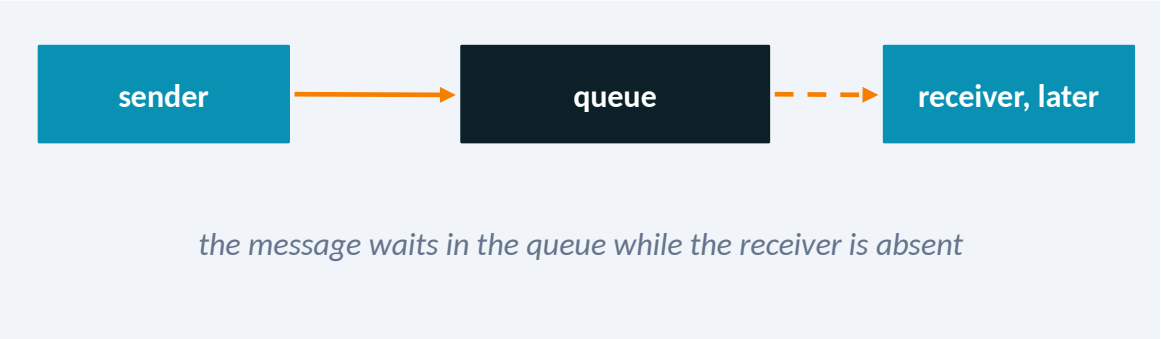
The message is stored by the system until the receiver collects it, so the receiver can start an hour later.

Why it matters for devices

A collector of sensor readings restarts and needs four minutes. With sockets every reading of those minutes is lost; with a queue they wait.

Real system

RabbitMQ and Amazon SQS are message queues.



Operation	What it does	Does it block the caller?
PUT	appends a message to the end of a named queue	no
GET	waits until the queue holds a message, then removes the first	yes, until one arrives
POLL	removes the first message if there is one, else returns empty	no
NOTIFY	registers a function that is called when a message arrives	no, the call comes later

A socket loses the message when the receiver is absent; a queue keeps it, at the cost of a store that must not lose what it holds.

A broker routes by name: a queue delivers once, a topic to all

Broker

A server that receives messages, stores them and delivers them. One RabbitMQ machine moves tens of thousands per second.

What it removes

The sender no longer needs the address of the receiver, and the two no longer run at the same time.

What it adds

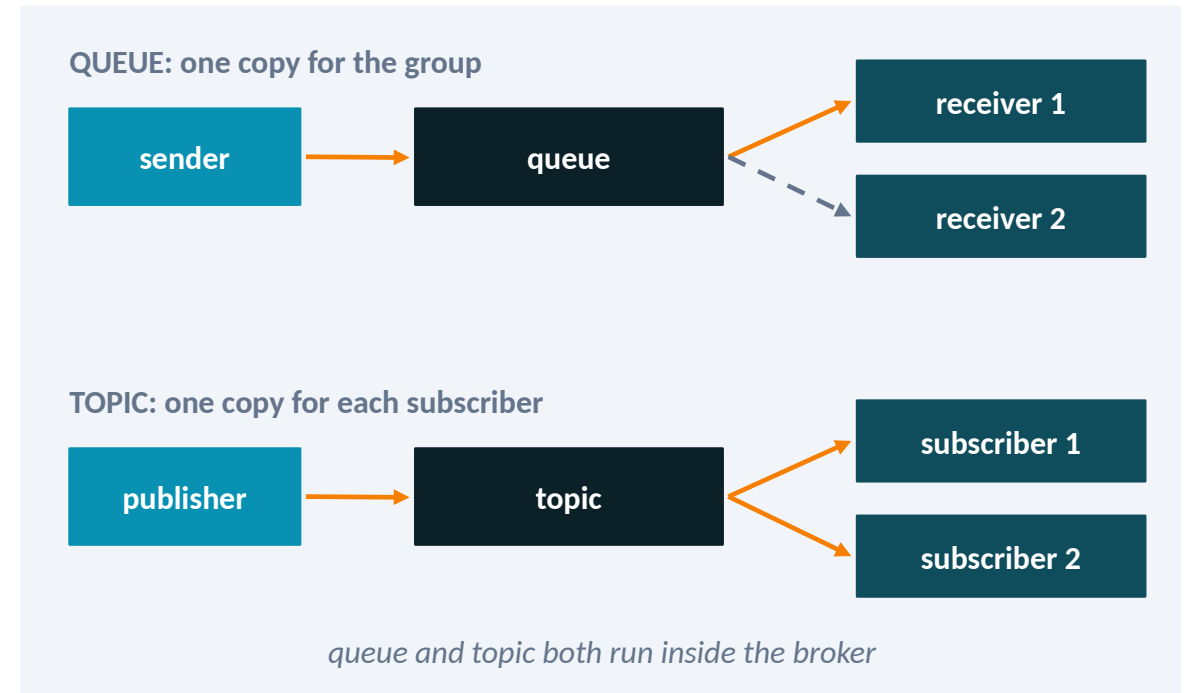
Routing by name, change of format between systems, and a store that survives a restart.

The risk

Every message crosses it, so it is the bottleneck and the component whose failure stops everything. Brokers run as several copies.

Queue against topic

A queue delivers each message to one receiver: workers share the load. A topic delivers a copy to every subscriber.



Real use

Order processing uses a queue: each order once. Price changes use a topic: every service that shows prices gets it.

Every message crosses the broker, so it is replicated. Work to be done once goes on a queue; a fact several services need goes on a topic.

Publish and subscribe: + matches one level, # matches all below

The model

A publisher sends a message to a named topic. Every program subscribed to that topic receives a copy, and a new subscriber changes nothing in the publisher.

A topic is a path

Levels separated by a slash: city/noise/sensor7 is the topic of one sensor, city/temp/sensor2 is another.

Two wildcards in a filter

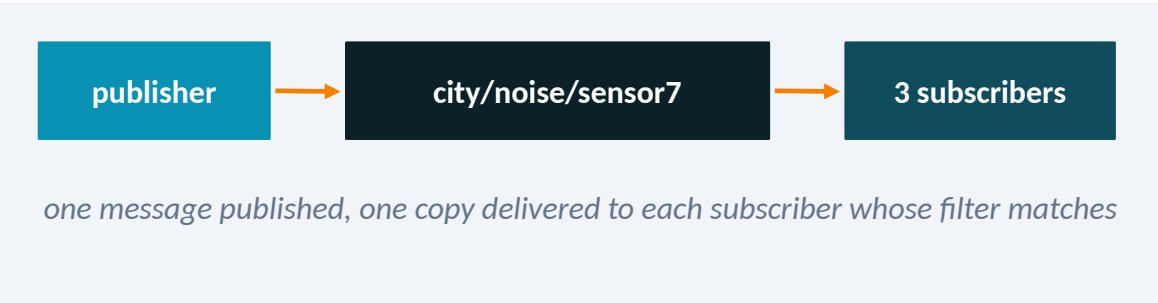
The sign + matches exactly one level. The sign # matches every level below, and it can only be the last element of the filter.

Real system

A smart meter publishes to one topic per meter, and the utility subscribes with one filter that covers every meter.

Message published to	city/noise/+	city/#	city/noise/sensor7
city/noise/sensor7	yes	yes	yes
city/noise/sensor7/raw	no: extra level	yes	no
city/temp/sensor2	no: temp	yes	no
city/noise	no: level missing	yes	no

each column is one subscriber with its filter; yes means the subscriber receives the message



The publisher names a topic and never learns who received the message; a subscriber names a filter and receives every topic that matches it.

MQTT adds a delivery guarantee: one, two or four messages

MQTT

The publish and subscribe model of the previous slide, with the same topics and wildcards, written for devices with little memory and a poor network. TCP, port 1883, broker in the middle.

Level 0, at most once

The device sends the message and forgets it: one message. If the network drops it, it is lost.

Level 1, at least once

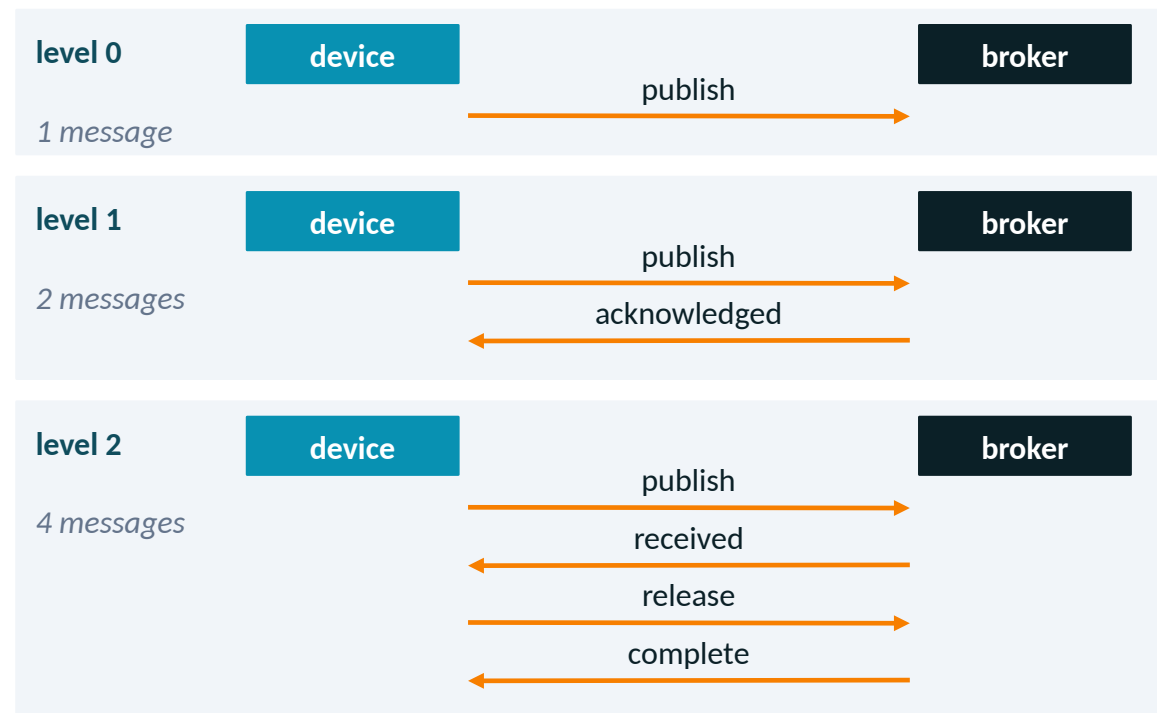
The device repeats until the broker confirms: two messages. A lost confirmation makes it arrive twice, so the receiver tolerates duplicates.

Level 2, exactly once

Four messages: publish, received, release, complete. Delivered once.

The last will

A device declares at connection a message that the broker publishes if the device disappears: a sensor is reported as dead this way.



Choosing

A battery sensor reporting every second uses level 0; a command that opens a valve uses level 2.

Level 0 may lose the message, level 1 may deliver it twice, level 2 delivers it once: 1, 2 and 4 messages per publication.

Multicast copies at the router; four mechanisms, four couplings

Multicast

One message delivered to a group, sent once by the source. Routers make the copies, so one copy crosses each link. One message per receiver would put a thousand copies on the first link.

Coupling in space

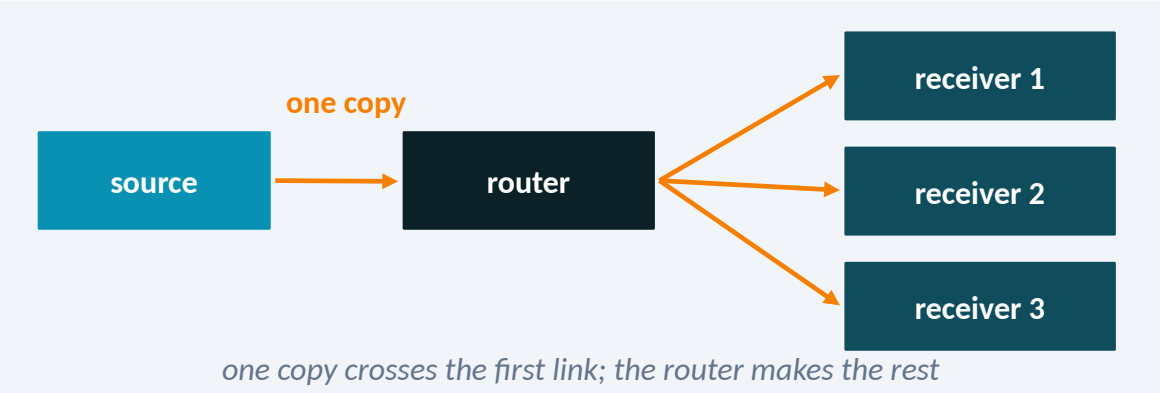
Two programs are coupled in space when each one must know the identity of the other.

Coupling in time

They are coupled in time when both must run at the same moment.

Real system

Internet television is delivered by multicast inside the network of the operator.



Mechanism	Must know the other	Both running	Coupling
socket	yes	yes	space and time
message queue	yes, queue name	no	space only
multicast	no	yes	time only
publish/subscribe, stored	no	no	neither

Copies made late cost less than copies made early; removing a coupling adds a component in the middle that must run.

BLOCK 2. ELASTICITY AND SCALABILITY

2.4 Scalability in the cloud

Machines are added and removed while the service runs. One component sends each request to a machine that is running, one loop sets how many machines there are, and one formula sizes what each machine needs.

CONTENTS

- 01 Scale up and scale out
- 02 Elasticity
- 03 The load balancer
- 04 The autoscaling loop
- 05 A queue in front
- 06 Little's law
- 07 Cache layers
- 08 Gossip membership
- 09 What breaks first

Scale up to the largest machine, or scale out behind one address

Scale up

Replace the machine with a larger one: 4 cores, then 16, then 64. The software does not change, and there is always a largest machine.

Scale out

Add machines of the same size behind one address. No limit in principle, and the software has to be written for it.

The condition

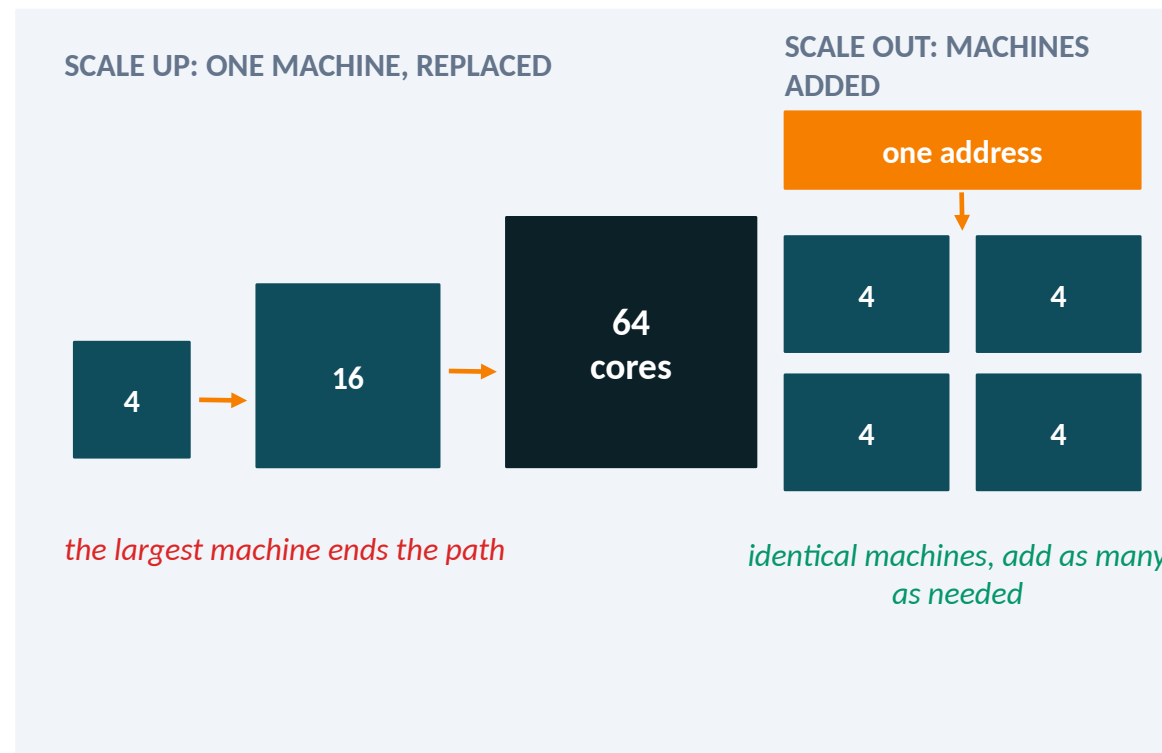
Any machine must be able to serve any request. A server that keeps the state of its clients in memory breaks it.

The usual path

Scale up while it is cheap, then scale out. The change is a rewrite, so it is planned early.

Real systems

A relational database usually scales up. A web front end usually scales out.



The number inside each box is the number of processor cores of that machine.

Scaling out works only when any machine can serve any request, so the servers keep no client state.

Elasticity: 33 machine hours instead of 72 for the same day

Elasticity

The machines running follow the demand: they appear in minutes when the load rises and are released when it falls. Rented capacity is paid by the hour, so the bill follows the same curve.

Scalability is a different property

A scalable system can grow. An elastic system grows and shrinks in minutes, with nobody intervening.

The numbers in the figure

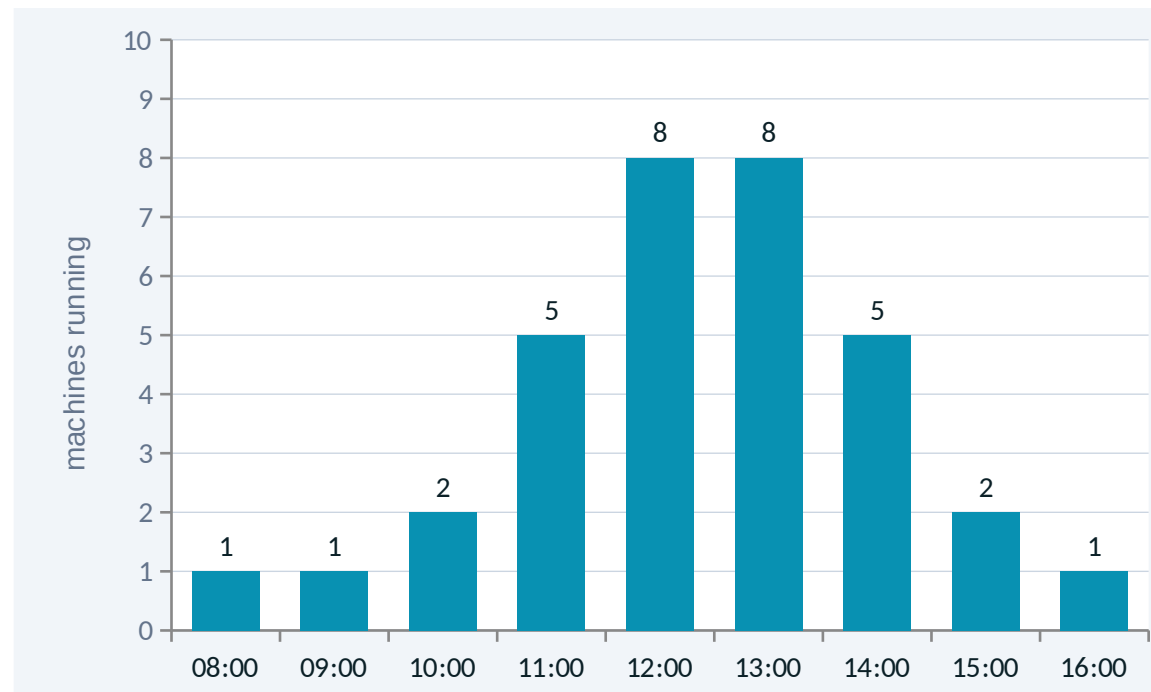
A ticket service for a concert on sale at ten. Machines per hour: 1, 1, 2, 5, 8, 8, 5, 2, 1: 33 machine hours. Eight all day would be 72.

What it requires

A machine must join and leave with nobody noticing, so the servers keep no client state.

Real system

Amazon EC2 rents machines by the hour and starts one in about a minute.



Eight machines at noon, one at four in the afternoon.

Elastic means the machines running follow the load within minutes; a fixed size for the worst hour pays that size all day.

The load balancer sends each request to a machine that is running

Goal

Send each request to a machine that is running, while machines are added and removed.

Structure

One address, and a list of the machines behind it with their state, healthy or failed, and their number of open connections.

Steps

- 1 every 10 s send a health check; two failures in a row mark the machine failed.
- 2 a request arrives: choose among the healthy ones, in turn or the one with fewest connections.
- 3 forward the request.

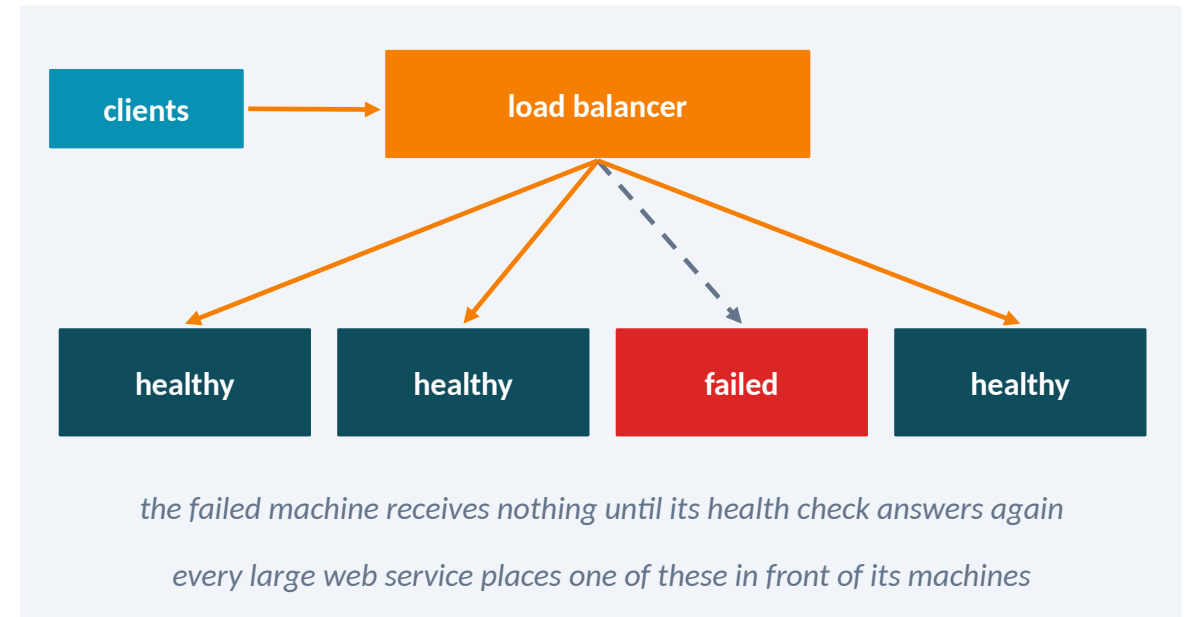
The obstacle

If a server keeps the session of a client in memory, every request of that client must reach that machine: the load cannot be spread.

The repair

Move the session to a shared store, Redis, or into the client.

The balancer spreads requests and skips failed machines; sessions move to a shared store so that any machine can answer.



Real system

nginx and HAProxy are load balancers; the balancer is itself duplicated, because every request crosses it.

Autoscaling loop: measure, compare, act, cool down

Steps

- 1 measure the processor use every minute.
- 2 compare: above 70 per cent for 3 minutes.
- 3 act: add one machine.
- 4 wait 5 minutes, so the loop does not react to its own change.

Removing machines

A lower threshold, below 30 per cent for 10 minutes, so the loop does not oscillate.

Cold start

A new machine is not useful at once: boot 20 s, download 40 s, cache warm 60 s, first useful request at 90 s.

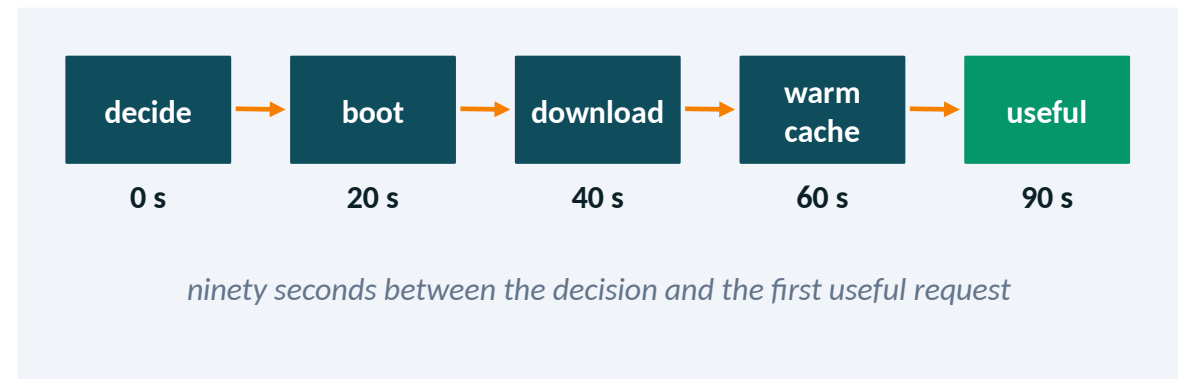
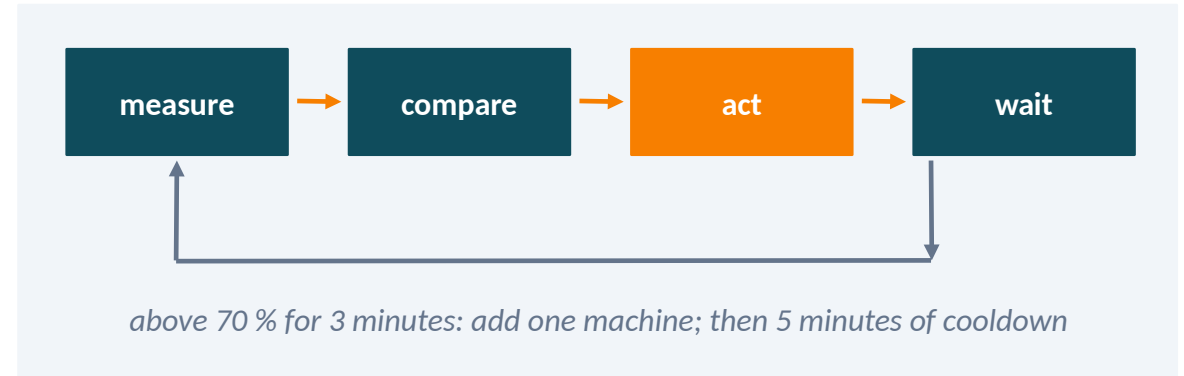
Worked case

Demand doubles in 30 s. Three minutes of measurement plus 90 s of start: 270 s at twice the load.

Two repairs

Keep a minimum running at all hours, or start machines before a known event: a sale at ten starts them at nine.

Demand that rises faster than the start time is not covered; a minimum of machines and a start before a known event cover it.



A queue in front: the backlog grows at arrival minus service

Structure

Producers write every request into a queue; a pool of workers reads it at its own rate.

Formulas

growth = $\lambda - \mu = 500 - 100 = 400$ per second.

backlog after 10 s = 4000 requests.

drain time = $4000 / 100 = 40$ s once the arrivals stop.

Steps for the operator

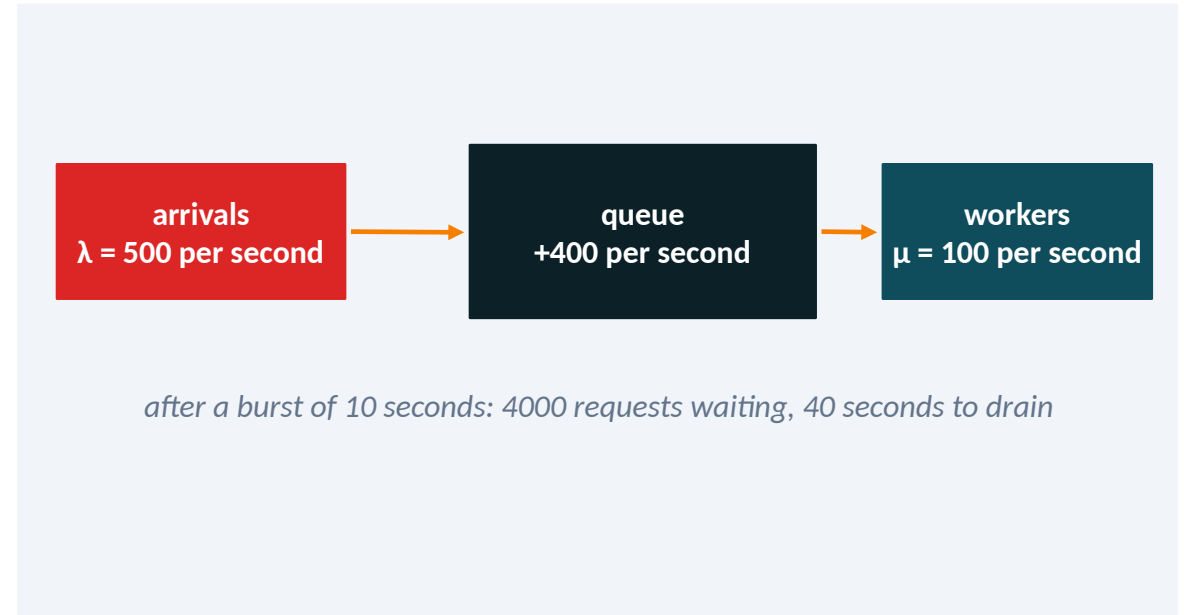
- 1 measure the length of the queue every minute.
- 2 if it grows for 1 minute, add a worker.
- 3 if it stays empty for 10 minutes, remove one.

When it is acceptable

For a burst that ends, because the workers catch up. With λ above μ as a steady state the queue grows without limit.

Real use

Orders on a sale day are accepted into a queue and confirmed by mail minutes later.

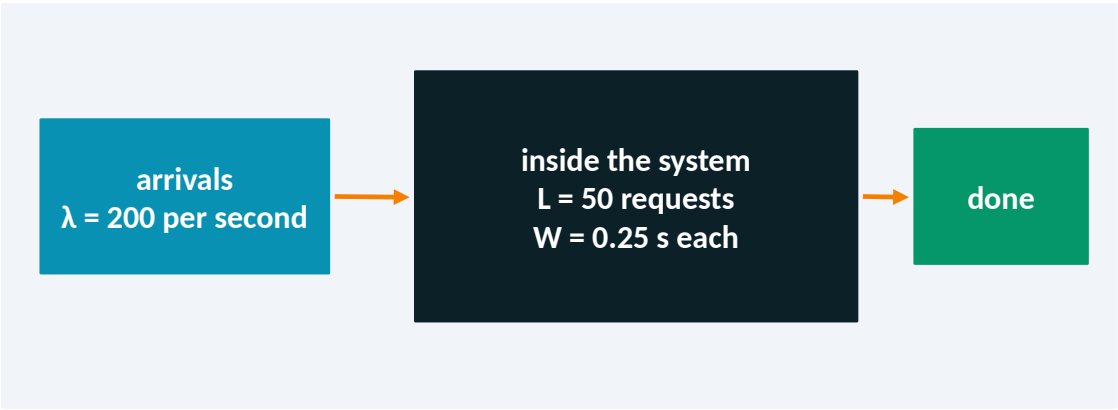


The queue grows during the burst and the workers never fail.

The autoscaler measures the length of the queue, because the queue grows before the users notice anything.

Little's law $L = \lambda W$: 200 per second $\times$ 0.25 s = 50 inside

The law: $L = \lambda \times W$. L requests inside the system at one moment, λ arrivals per second, W seconds each request spends inside. It holds in a steady state, because the requests inside are the ones that arrived during the last W seconds.



Direction	Given	Computation	Result
size the pool	$\lambda = 200$ per second, $W = 0.25$ s	$L = 200 \times 0.25$	50 inside: at least 50 threads or connections
find the limit	pool of 20 connections, $W = 0.25$ s	$\lambda = L / W = 20 / 0.25$	80 requests per second at most, whatever the processor does

Who does it and when

The operator, before deploying: 1 measure λ at the peak hour. 2 measure W . 3 compute $L = \lambda W$. 4 set every pool at or above L .

What it sizes

Thread counts and connection pools: worker_connections in nginx, max_connections in PostgreSQL, and the size of a queue.

Requests inside the system equal the arrival rate times the time each one spends inside; a pool smaller than L becomes the limit.

Cache layers: 80 per cent hits on a 40 ms database gives 8 ms

The principle

Each layer answers what it holds and passes the rest on. Only the misses reach the layer behind.

Formula

average = miss ratio $\times$ cost. $0.2 \times 40 = 8$ ms. To halve it to 4 ms the miss ratio must be 0.1, that is a hit ratio of 90 per cent.

The four layers

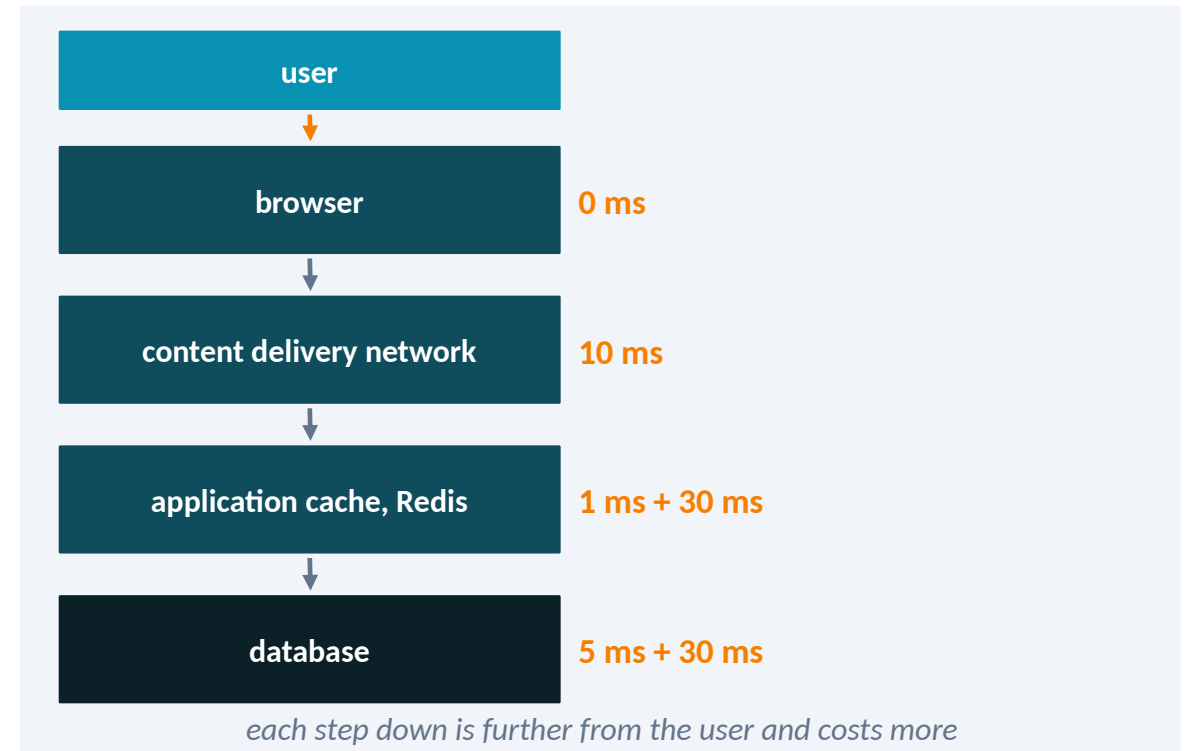
The browser keeps a file with an expiry time. The delivery network keeps a copy inside the network of the operator. The application cache is a shared store of results, Redis.

The rule every layer needs

How long an answer may be kept. In HTTP it is Cache-Control: max-age=300, five minutes.

Real use

A product page is built from the four layers; only the stock figure reaches the database.



The cheapest request never travels; each layer answers what it can and needs a rule for how long it may keep an answer.

EACH SERVER MUST LEARN WHICH SERVERS ARE RUNNING

Gossip: one random exchange per second, 100 servers in 7 s

What is alive here

A server process of the service that answers. Not a person and not a machine: a machine can run and its process be dead.

Structure

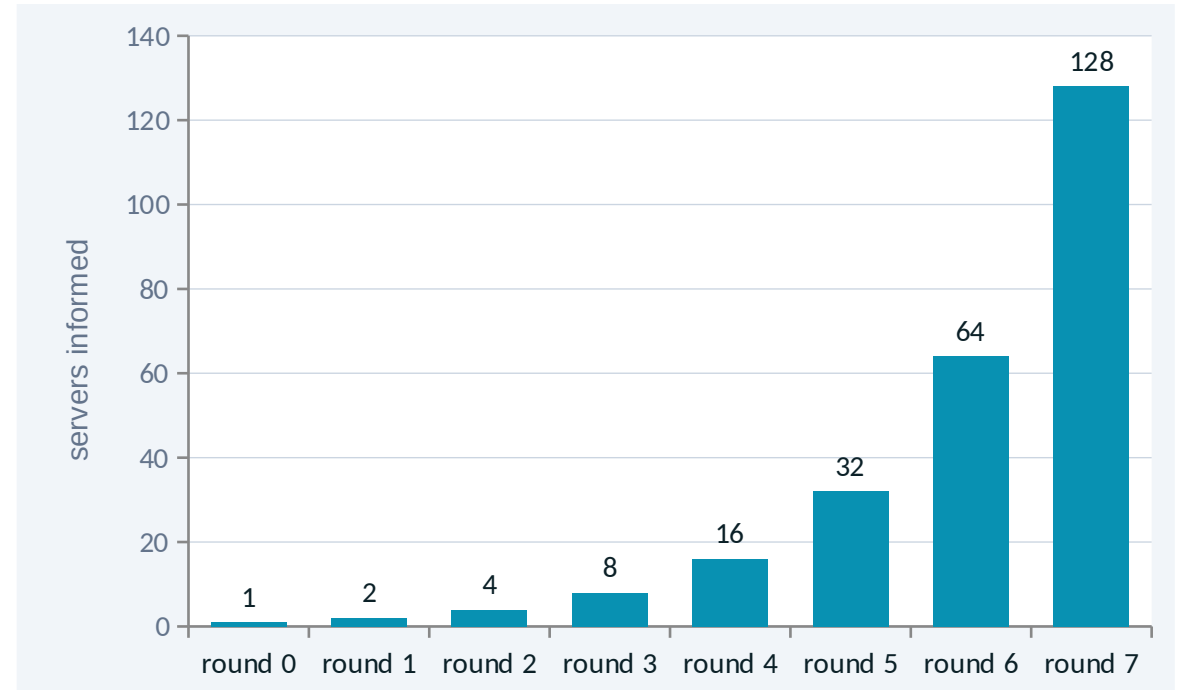
Each server keeps a list with one row per member: address, a counter, and the moment the counter last grew. Each server raises its own counter every second.

Steps

- 1 every second pick one member at random.
- 2 send it your list.
- 3 merge the two lists, keeping the higher counter of each member.
- 4 a counter that has not grown for 10 s marks that member as failed.

Why it is fast, and its limit

The informed count doubles every round: 1, 2, 4, 8, 16, 32, 64, 128. After 7 rounds 128 passes 100: about 7 seconds. No moment marks the fact as fully spread: speed and no guarantee.



The count doubles every round; 128 passes 100 at round 7.

Real system

Cassandra uses gossip so that every node holds the state of its ring.

Gossip spreads a fact in about $\log_2 N$ rounds with no central register, and no moment marks the fact as known to everyone.

The capacity is the component that every request touches

The numbers in the figure

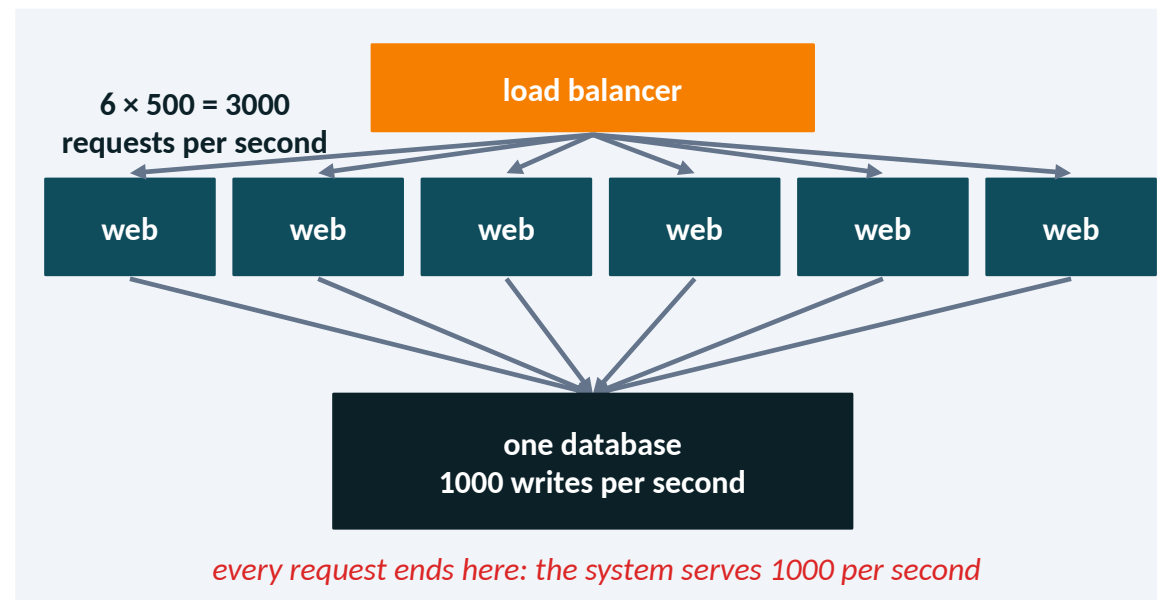
Six web machines serve 500 requests per second each: 3000 in total. The database accepts 1000 writes per second. The system serves 1000.

Why not the front

Web machines keep no client state and multiply behind the balancer, so they are almost never the limit.

Four repairs, in this order

- 1 cache the reads, so fewer requests reach the database.
- 2 move the writes to a queue, so the database receives them at its own rate.
- 3 divide the data into shards, so writes spread over several primaries.
- 4 accept an old value where the business allows it.



The same rule on a city measurement service

Two thousand sensors publish one reading per minute to a Mosquitto broker: 33 per second, concentrated at the full minute. A gateway near the sensors groups them, a RabbitMQ queue absorbs the peak, workers are added with the length of that queue, and Cassandra stores the readings divided by sensor identifier with three copies. Without the queue the 2000 readings arrive at the database at once.

Find the component every request touches: its rate is the rate of the whole system, whatever the rest can do.